

Complex Processing Issues: Codebase Deep Analysis

本ドキュメントは、CV-MsAtViT (Complex-Valued Multi-scale Attention ViT) の実装コードを詳細に解析し、複素数処理における**「実部と虚部の分離処理 (Split-Complex Processing)」と、それが引き起こす「位相情報の歪み・喪失」**について、具体的なコード箇所と計算式に基づいて記述したものです。

1. Multi-scale Feature Extractor (Conv3D)

概要: 入力されたPolSARデータから特徴を抽出する初期段階。

コード箇所: `model_factory.py` : `MultiScaleFeatureExtractor` 関数 (L61-108)

使用状況

3つの異なるカーネルサイズ $((3,3,1), (1,1,3), (3,3,3))$ を持つ `ComplexConv3D` ブロックが並列に定義されていますが、すべての畳み込み層において活性化関数として `cart_relu` が指定されています。

```
x1 = ComplexConv3D(..., activation="cart_relu", ...)(inputs) # L65
# ... (他5箇所のConv3Dすべて同様)
```

計算式 (Cartesian ReLU)

$$\text{cart_relu}(z) = \text{ReLU}(\text{Re}(z)) + j \cdot \text{ReLU}(\text{Im}(z))$$

- 物理的意味:** 複素平面の第3象限（実部負・虚部負）にある値をゼロにします。
- 問題点:** 位相角が -135° 付近の情報が完全に欠落し、それ以外の領域でも「振幅を変えずに位相だけ回す」等の操作ができず、位相空間を正方形にクリッピングします。

2. Reshape & 2D Projection

概要: 3次元の特徴マップを2次元パッチに変換する前段階。

コード箇所: `model_factory.py`: `build_msatvit` 関数 (L188)

使用状況

`Reshape` (L187) で次元を整理した後、チャンネル数を調整するための `ComplexConv2D` 層で `cart_relu` が使用されています。

```
x = ComplexConv2D(..., activation="cart_relu", ...)(x) # L188
```

計算式

(同上)

$$z_{out} = \text{ReLU}(\text{Re}(z_{in} \cdot W + b)) + j \cdot \text{ReLU}(\text{Im}(z_{in} \cdot W + b))$$

3. Coordinate Attention (Bottleneck / 1x1 Conv)

概要: 特徴マップの水平・垂直方向の圧縮情報を統合するボトルネック部分。

コード箇所: `CoordAttention.py`: `CoordAtt_cmplx` 関数 (L44), `coord_act` 関数 (L30-33)

使用状況

`ComplexConv2D` (L42) と `ComplexBatchNormalization` (L43) を通過した後、カスタム活性化関数 `coord_act` が適用されています。これは `h_swish` を模した関数です。

```
y = coord_act(y) # L44
```

計算式 (Asymmetric H-Swish)

$$\text{coord_act}(z) = z \times \frac{\text{cart_relu}(z + 3)}{6}$$

ここで $z + 3$ は実部のみに加算されるため $(\text{Re}(z) + 3) + j\text{Im}(z)$ となります。

したがって、ゲート項 $G = \frac{\text{cart_relu}(z+3)}{6}$ は以下ようになります：

$$\text{Re}(G) = \frac{\text{ReLU}(\text{Re}(z) + 3)}{6}, \quad \text{Im}(G) = \frac{\text{ReLU}(\text{Im}(z))}{6}$$

- **物理的意味:**
 - $\text{Im}(z) < 0$ の場合、 $\text{Im}(G) = 0$ となり、ゲートは実数になります（純粋なスケールリング、位相保存）。
 - $\text{Im}(z) > 0$ の場合、 $\text{Im}(G) > 0$ となり、ゲートは複素数になります（スケールリング + 回転）。
- **問題点:** 虚部の符号によって回転するかどうかが変わるという、極めて非対称で物理的根拠のない位相歪みを引き起こします。

4. Coordinate Attention (Expansion)

概要: 統合された特徴を再び高さ(h)と幅(w)の注意重みマップに分離・展開する部分。

コード箇所: CoordAttention.py : CoordAtt_cmplx 関数 (L49-50)

使用状況

分離された特徴マップに対し、1x1畳み込みを行い、最終的なAttention Weightを生成する際に `cart_sigmoid` が使用されています。

```
a_h = ComplexConv2D(..., activation="cart_sigmoid")(x_h) # L49
a_w = ComplexConv2D(..., activation="cart_sigmoid")(x_w) # L50
```

計算式 (Cartesian Sigmoid / "First-Quadrant Trap")

$$\text{cart_sigmoid}(z) = \text{Sigmoid}(\text{Re}(z)) + j \cdot \text{Sigmoid}(\text{Im}(z))$$

- **物理的意味:**
 - $\text{Re}(\text{Out}) \in (0, 1)$
 - $\text{Im}(\text{Out}) \in (0, 1)$
- **問題点 (第1象限の罠):** 出力される重み係数の実部と虚部が常に正になります。これは位相角が常に $0^\circ \sim 90^\circ$ (第1象限) に限定されることを意味します。モデルは位相を「進める」ことはできても、「遅らせる (第4象限)」や「反転させる (第2・3象限)」補正を行うことが数学的に不可能です。

5. Complex ViT: Normalization & Attention Inputs

概要: Transformerブロックの核心部分。

コード箇所: `model_factory.py` : `cmplx_ViT` 関数内ループ (L125-134)

使用状況

Self-Attentionに入力する直前で、データが実部と虚部に**分離**され、**独立に Layer Normalization** され、**独立に Attention** に入力されます。

1. 独立正規化 (L125-126)

```
x1_r = layers.LayerNormalization(...)(tf.math.real(encoded_patches))
x1_i = layers.LayerNormalization(...)(tf.math.imag(encoded_patches))
```

2. 独立Attention (L128-133)

```
attention_output_r = MultiHeadAttention(...)(x1_r, x1_r)
attention_output_i = MultiHeadAttention(...)(x1_i, x1_i)
```

計算式 (Split Processing)

本来あるべき姿は $Z' = \text{ComplexLayerNorm}(Z)$ ですが、実際に行われているのは：

$$\text{Re}(Z') = \frac{\text{Re}(Z) - \mu_r}{\sigma_r}, \quad \text{Im}(Z') = \frac{\text{Im}(Z) - \mu_i}{\sigma_i}$$

Split-Attention の詳細計算フロー:

ここで実部と虚部は「全く別々のデータ」として扱われます。

1. 実部チーム (Real Path):

- $Q_r = x1_r W_{Qr}, \quad K_r = x1_r W_{Kr}, \quad V_r = x1_r W_{Vr}$
- $\text{Score}_r = \text{Softmax} \left(\frac{Q_r K_r^T}{\sqrt{d}} \right)$ (実部だけの重要度マップ)
- $C_r = \text{Score}_r V_r$ (実部だけの加重平均)
- $O_r = C_r W_{Or}$ (最後の混ぜ合わせ)

2. 虚部チーム (Imaginary Path):

- $Q_i = x1_i W_{Qi}, \quad K_i = x1_i W_{Ki}, \quad V_i = x1_i W_{Vi}$
- $\text{Score}_i = \text{Softmax} \left(\frac{Q_i K_i^T}{\sqrt{d}} \right)$ (虚部だけの重要度マップ)

- $C_i = \text{Score}_i V_i$ (虚部だけの加重平均)
- $O_i = C_i W_{O_i}$ (最後の混ぜ合わせ)

3. 再結合 (Recombine) (L134):

$$O = \text{tf.complex}(O_r, O_i)$$

ここでようやく複素数に戻ります。

- **問題点:** 実部と虚部でそれぞれ異なる重要度マップ ($\text{Score}_r \neq \text{Score}_i$) を持っているため、同一画素に対しても「実部は重要視するが虚部は無視する」といった**インコヒーレント (不干涉) な状態**が発生します。クロス項 ($Q_r K_i^T$ など) が一切計算されないため、位相差に基づく相関関係は完全に無視されます。

6. Complex ViT: Normalization & MLP (Complex Dense)

概要: Transformerブロック内のFeed Forward Network部分。

コード箇所: `model_factory.py`: `cmplx_ViT` (L138-143), `cmplx_multilayer_perceptron` (L18-22), `SAR_utils.py` (`cart_gelu`)

使用状況

MLPに入力する直前でも、再度 **独立 Layer Normalization** が行われます。その後、MLP内部では `ComplexDense` が適用され、活性化関数として `cart_gelu` が使用されます。

1. 独立正規化 (Pre-MLP) (L138-139)

```
x3_r = layers.LayerNormalization(...)(tf.math.real(x2))
x3_i = layers.LayerNormalization(...)(tf.math.imag(x2))
```

2. MLP内部 (Complex Interaction with Split Activation) (L20)

```
x = ComplexDense(units, activation=cart_gelu)(x)
```

計算式 (Complex Dense + Cartesian GELU)

$$Z_{mid} = Z_{in} \cdot W + b \quad (\text{これは正しい複素行列積})$$

$$Z_{out} = \text{cart_gelu}(Z_{mid}) = \text{GELU}(\text{Re}(Z_{mid})) + j \cdot \text{GELU}(\text{Im}(Z_{mid}))$$

※**GELU (Gaussian Error Linear Unit) とは:**

ReLU ($x < 0$ で0, $x \geq 0$ でそのまま) を滑らかにした関数です。

$$\text{GELU}(x) = x\Phi(x) \approx 0.5x \left(1 + \tanh \left[\sqrt{2/\pi}(x + 0.044715x^3) \right] \right)$$

- 負の値に対しても完全なゼロにはならず、わずかに負の値を許容してからゼロに近づきます。
- これも「負の値を抑制する」性質があるため、複素平面上で適用すると、実部・虚部がそれぞれ独立に「抑制」または「通過」され、位相角が大きく歪みます。
- **物理的意味:**
 - ComplexDense 部分では、実部と虚部のクロス計算 ($\text{Re} \cdot \text{Im}$ 等) が行われ、唯一まともな相互作用が発生します。
 - しかし、その直後に適用される `cart_gelu` によって、再び実部と虚部は独立に非線形変換を受けます。GELUは負の値を抑制するため、実部・虚部がそれぞれ独立に抑制・強調され、結果として位相が歪みます。